ECG Rhythm Classification

The goal of this exercise is to train a neural network model to classify different cardiac rhythm from single-lead ECG signals. The ECG signals we will use are a subset of the large scale 12-lead electrocardiogram database for arrhythmia study (https://physionet.org/content/ecg-arrhythmia/1.0.0/).

The subset includes the following cardiac rhythms:

- Atrial fibrillation
- Atrial flutter
- Normal sinus rhythm
- Sinus bradycardia
- Sinus tachycardia

There are 1500 single-lead ECG signals (lead II) for each rhythm.

Group Members: Mamoun Alaoui Slimani, Léa Slive, Yasaman Noorikhah, Antonio Del Priore Antunes
Date of Submission: Thursday, October 2, 2025

Please find the answer of each question written as a markdown or code cell after the question.

First, we import all required packages, define global constants, and seed the random number generators to obtain reproducible results.

In [1]:
```
%matplotlib widget


import collections
import itertools
import logging
import operator
import pathlib
import warnings

import IPython.display
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import pytorch_lightning as pl
import sklearn.metrics
import sklearn.model_selection
import sklearn.preprocessing
import torch


DATA_FILE = pathlib.Path('../data/ecg_rhythms.npz')
LOG_DIRECTORY = pathlib.Path('../logs/ecg_rhythm_classification')
```

```
# Disable logging for PyTorch Lightning to avoid too long outputs.
logging.getLogger('pytorch_lightning').setLevel(logging.ERROR)

# Seed random number generators for reproducible results.
pl.seed_everything(42)
```

```
Seed set to 42
```

Out[1]: 42

Then, we load the ECG signals and the corresponding rhythm annotations.

In [2]:
```
def load_data():
    with np.load(DATA_FILE) as data:
        signals = data['signals']
        rhythms = data['rhythms']
        fs = data['fs'].item()
    return signals, rhythms, fs


signals, rhythms, fs = load_data()

IPython.display.display(pd.DataFrame(sorted(collections.Counter(rhythms).items()
```

	rhythm	count
0	atrial_fibrillation	1500
1	atrial_flutter	1500
2	normal_sinus_rhythm	1500
3	sinus_bradycardia	1500
4	sinus_tachycardia	1500

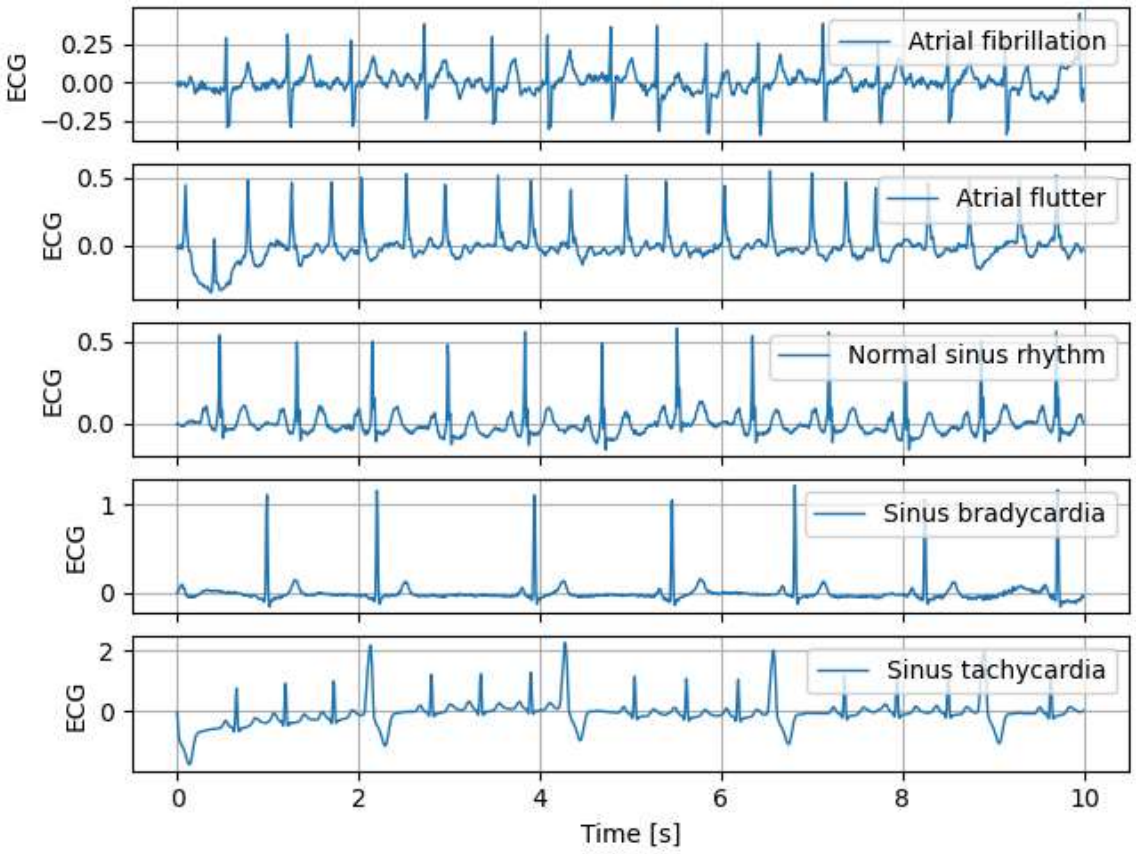
Here are a few examples of ECG signals.

In [3]:
```
def plot_ecg_examples(signals, rhythms, fs):
    time = np.arange(signals.shape[-1]) / fs
    labels = np.unique(rhythms)
    indices = [np.random.choice(np.flatnonzero(rhythms == label)) for label in l

    fig, axes = plt.subplots(len(labels), 1, sharex='all', constrained_layout=Tr
    for ax, index in zip(axes.flat, indices):
        label = rhythms[index].replace('_', ' ').capitalize()
        plt.sca(ax)
        plt.plot(time, signals[index].T, linewidth=1, label=label)
        plt.grid()
        plt.ylabel('ECG')
        plt.legend(loc='upper right')
    plt.xlabel('Time [s]')


plot_ecg_examples(signals, rhythms, fs)
```

Figure



Question 1

Visually, what are the differences between the different rhythms?

Answer: Sinus rhythms are regular and narrow complexes: bradycardia is slower, tachycardia faster, both with clear P waves and consistent RR intervals. Normal sinus sits in between. Atrial fibrillation is irregularly irregular with no consistent P waves and varying amplitude. Atrial flutter shows rapid, sawtooth-like atrial activity with a more regular but faster rate than sinus rhythm.

Then, we split that data into subsets for training, validation, and testing stratified by rhythms.

In [4]:
```
def split_data(rhythms):
    n = rhythms.size
    splitter = sklearn.model_selection.StratifiedKFold(n_splits=5)
    indices = list(map(operator.itemgetter(1), splitter.split(np.zeros((n, 1)),
    i_train = np.hstack(indices[:-2])
    i_val = indices[-2]
    i_test = indices[-1]

    assert np.intersect1d(i_train, i_val).size == 0
    assert np.intersect1d(i_train, i_test).size == 0
    assert np.intersect1d(i_val, i_test).size == 0
    assert np.all(np.sort(np.hstack((i_train, i_val, i_test))) == np.arange(n))

    return i_train, i_val, i_test
```

```
i_train, i_val, i_test = split_data(rhythms)


def build_summary(rhythms, indices):
    labels = np.unique(rhythms)
    data = []
    for subset, i in indices:
        y = rhythms[i]
        data.append({'subset': subset, 'total_count': y.size})
        for label in labels:
            data[-1][f'{label}_count'] = np.sum(y == label)
    return pd.DataFrame(data)


IPython.display.display(build_summary(rhythms, (('train', i_train), ('val', i_va
```

	subset	total_count	atrial_fibrillation_count	atrial_flutter_count	normal_sinus_rhythm_c
0	train	4500	900	900	
1	val	1500	300	300	
2	test	1500	300	300	

The final preprocessing steps are to scale the ECG signals so that they have approximiately unit variance and to encode the rhythm labels with one-hot encoding.

In [5]:
```
def compute_scaling(signals):
    sigma = np.std(signals)
    return 1.0 / sigma


alpha = compute_scaling(signals[i_train])
signals *= alpha


def encode_rhythms(rhythms):
    categories = [np.unique(rhythms)]
    encoder = sklearn.preprocessing.OneHotEncoder(categories=categories, sparse_
    return encoder.fit_transform(rhythms[:, None])


encoded_rhythms = encode_rhythms(rhythms)


def print_encoded_rhythms(rhythms, encoded_rhythms, n=10):
    df = pd.DataFrame(encoded_rhythms, columns=np.unique(rhythms))
    df.insert(0, 'rhythm', rhythms)
    IPython.display.display(df.head(n))


print_encoded_rhythms(rhythms, encoded_rhythms)
```

	rhythm	atrial_fibrillation	atrial_flutter	normal_sinus_rhythm	sinus_bradyca
0	atrial_fibrillation	1.0	0.0	0.0	
1	sinus_bradycardia	0.0	0.0	0.0	
2	sinus_bradycardia	0.0	0.0	0.0	
3	atrial_flutter	0.0	1.0	0.0	
4	sinus_bradycardia	0.0	0.0	0.0	
5	atrial_fibrillation	1.0	0.0	0.0	
6	normal_sinus_rhythm	0.0	0.0	1.0	
7	sinus_bradycardia	0.0	0.0	0.0	
8	sinus_bradycardia	0.0	0.0	0.0	
9	sinus_bradycardia	0.0	0.0	0.0	

We define a class and few utility functions for training and evaluating models.

In [6]:
```
class Classifier(pl.LightningModule):

    def __init__(self, model, learning_rate=0.001):
        super().__init__()
        self.save_hyperparameters(ignore=['model'])
        self.model = model
        self.learning_rate = learning_rate
        self.example_input_array = torch.zeros((1,) + self.model.input_shape)

    def configure_optimizers(self):
        return torch.optim.Adam(self.parameters(), lr=self.learning_rate)

    def forward(self, x):
        return self.model(x)

    def training_step(self, batch, batch_idx):
        return self._run_step(batch, 'train')

    def validation_step(self, batch, batch_idx):
        self._run_step(batch, 'val')

    def test_step(self, batch, batch_idx):
        self._run_step(batch, 'test')

    def predict_step(self, batch, batch_idx, dataloader_idx=0):
        x, y = batch
        return self.model(x)

    def _run_step(self, batch, subset):
        x, y = batch
        logits = self.model(x)
        loss = torch.nn.functional.cross_entropy(logits, y)
        acc = (torch.argmax(y, 1) == torch.argmax(logits, 1)).float().mean()
        self.log_dict({
            f'{subset}_loss': loss,
            f'{subset}_acc': acc,
```

```
        }, on_step=False, on_epoch=True, prog_bar=True)
        return loss


def build_loader(*tensors, batch_size=100, shuffle=False, n_workers=0):
    dataset = torch.utils.data.TensorDataset(*map(torch.Tensor, tensors))
    return torch.utils.data.DataLoader(
        dataset=dataset,
        batch_size=batch_size,
        shuffle=shuffle,
        num_workers=n_workers,
    )


def train_model(name, model, x, y, i_train, i_val, learning_rate=0.001, batch_si
    train_loader = build_loader(x[i_train], y[i_train], batch_size=batch_size, s
    val_loader = build_loader(x[i_val], y[i_val], batch_size=batch_size)
    classifier = Classifier(model, learning_rate)
    print(pl.utilities.model_summary.ModelSummary(classifier, max_depth=-1))

    with warnings.catch_warnings():
        warnings.simplefilter('ignore')
        trainer = pl.Trainer(
            default_root_dir=LOG_DIRECTORY,
            logger=pl.loggers.TensorBoardLogger(LOG_DIRECTORY, name),
            enable_model_summary=False,
            max_epochs=n_epochs,
        )
        trainer.fit(classifier, train_loader, val_loader)

    return classifier


def evaluate_model(model, x, y, i_train, i_val, i_test, batch_size=100):
    loader = build_loader(x, y, batch_size=batch_size)

    with warnings.catch_warnings():
        warnings.simplefilter('ignore')
        trainer = pl.Trainer(
            default_root_dir=LOG_DIRECTORY,
            logger=False,
            enable_progress_bar=False,
            enable_model_summary=False,
        )
        z = trainer.predict(model, loader)
    z = np.vstack([u.numpy() for u in z])

    references = np.argmax(y, axis=1)
    predictions = np.argmax(z, axis=1)
    matrices = {}
    for subset, indices in (('train', i_train), ('val', i_val), ('test', i_test)
        matrices[subset] = sklearn.metrics.confusion_matrix(
            references[indices],
            predictions[indices],
        )

    return matrices
```

We start TensorBoard to visualize performance metrics during training.

If you prefer to view TensorBoard in a separate window, you can open http://localhost:6006/ in your web browser.

In [7]:
```
%reload_ext tensorboard
%tensorboard --logdir ../logs/ecg_rhythm_classification --port 6006
```

```
Reusing TensorBoard on port 6006 (pid 29372), started 0:30:34 ago. (Use '!kill 29
372' to kill it.)
```



We define a convolutional neural network.

In [8]:
```
class CnnModel(torch.nn.Module):

    def __init__(self, input_shape, output_shape, kernel_size=5):
        super().__init__()
        self.input_shape = input_shape
        self.output_shape = output_shape
        self.layers = torch.nn.Sequential(
            torch.nn.Conv1d(self.input_shape[0], 8, kernel_size, padding='same'),
            torch.nn.BatchNorm1d(8),
            torch.nn.ReLU(),
            torch.nn.MaxPool1d(2),

            torch.nn.Conv1d(8, 16, kernel_size, padding='same'),
            torch.nn.BatchNorm1d(16),
            torch.nn.ReLU(),
            torch.nn.MaxPool1d(2),

            torch.nn.Conv1d(16, 32, kernel_size, padding='same'),
            torch.nn.BatchNorm1d(32),
            torch.nn.ReLU(),
            torch.nn.MaxPool1d(2),

            torch.nn.Conv1d(32, 64, kernel_size, padding='same'),
            torch.nn.BatchNorm1d(64),
            torch.nn.ReLU(),
            torch.nn.AdaptiveAvgPool1d(1),

            torch.nn.Flatten(),
            torch.nn.Linear(64, self.output_shape[0]),
        )

    def forward(self, x):
        return self.layers(x)
```

Then, we train and evaluate this model.

In [9]:
```
input_shape = signals.shape[1:]
output_shape = encoded_rhythms.shape[1:]
n_epochs = 50
batch_size = 100

cnn = train_model(
    name='cnn',
    model=CnnModel(input_shape, output_shape),
    x=signals,
    y=encoded_rhythms,
    i_train=i_train,
    i_val=i_val,
    learning_rate=0.0001,
    batch_size=batch_size,
    n_epochs=n_epochs,
)

cnn_matrices = evaluate_model(
    model=cnn,
    x=signals,
    y=encoded_rhythms,
    i_train=i_train,
    i_val=i_val,
```

```
    i_test=i_test,
    batch_size=batch_size,
)
```

```
   | Name            | Type              | Params | Mode  | FLOPs | In sizes
| Out sizes
-------------------------------------------------------------------------------
--------------------
0  | model           | CnnModel          | 14.2 K | train | 5.8 M | [1, 1, 1280]
| [1, 5]
1  | model.layers    | Sequential        | 14.2 K | train | 5.8 M | [1, 1, 1280]
| [1, 5]
2  | model.layers.0  | Conv1d            | 48     | train | 102 K | [1, 1, 1280]
| [1, 8, 1280]
3  | model.layers.1  | BatchNorm1d       | 16     | train | 0     | [1, 8, 1280]
| [1, 8, 1280]
4  | model.layers.2  | ReLU              | 0      | train | 0     | [1, 8, 1280]
| [1, 8, 1280]
5  | model.layers.3  | MaxPool1d         | 0      | train | 0     | [1, 8, 1280]
| [1, 8, 640]
6  | model.layers.4  | Conv1d            | 656    | train | 819 K | [1, 8, 640]
| [1, 16, 640]
7  | model.layers.5  | BatchNorm1d       | 32     | train | 0     | [1, 16, 640]
| [1, 16, 640]
8  | model.layers.6  | ReLU              | 0      | train | 0     | [1, 16, 640]
| [1, 16, 640]
9  | model.layers.7  | MaxPool1d         | 0      | train | 0     | [1, 16, 640]
| [1, 16, 320]
10 | model.layers.8  | Conv1d            | 2.6 K  | train | 1.6 M | [1, 16, 320]
| [1, 32, 320]
11 | model.layers.9  | BatchNorm1d       | 64     | train | 0     | [1, 32, 320]
| [1, 32, 320]
12 | model.layers.10 | ReLU              | 0      | train | 0     | [1, 32, 320]
| [1, 32, 320]
13 | model.layers.11 | MaxPool1d         | 0      | train | 0     | [1, 32, 320]
| [1, 32, 160]
14 | model.layers.12 | Conv1d            | 10.3 K | train | 3.3 M | [1, 32, 160]
| [1, 64, 160]
15 | model.layers.13 | BatchNorm1d       | 128    | train | 0     | [1, 64, 160]
| [1, 64, 160]
16 | model.layers.14 | ReLU              | 0      | train | 0     | [1, 64, 160]
| [1, 64, 160]
17 | model.layers.15 | AdaptiveAvgPool1d | 0      | train | 0     | [1, 64, 160]
| [1, 64, 1]
18 | model.layers.16 | Flatten           | 0      | train | 0     | [1, 64, 1]
| [1, 64]
19 | model.layers.17 | Linear            | 325    | train | 640   | [1, 64]
| [1, 5]
-------------------------------------------------------------------------------
--------------------
14.2 K    Trainable params
0         Non-trainable params
14.2 K    Total params
0.057     Total estimated model params size (MB)
20        Modules in train mode
0         Modules in eval mode
5.8 M     Total Flops
Output()
```

After the evaluation is finished, we can plot the confusion matrices for the training, validatoin, and test sets.

In [10]:
```
def plot_confusion_matrix(c, labels=None, title=None):
    c = np.asarray(c)

    fig = plt.figure(figsize=(5, 4), constrained_layout=True)
    image = plt.imshow(c, cmap='Blues', interpolation='nearest')

    threshold = (c.min() + c.max()) / 2
    for i, j in itertools.product(range(c.shape[0]), repeat=2):
        if c[i, j] < threshold:
            color = image.cmap(image.cmap.N)
        else:
            color = image.cmap(0)
        text = format(c[i, j], '.2g')
        if c.dtype.kind != 'f':
            integer_text = format(c[i, j], 'd')
            if len(integer_text) < len(text):
                text = integer_text
        plt.text(j, i, text, color=color, ha='center', va='center')

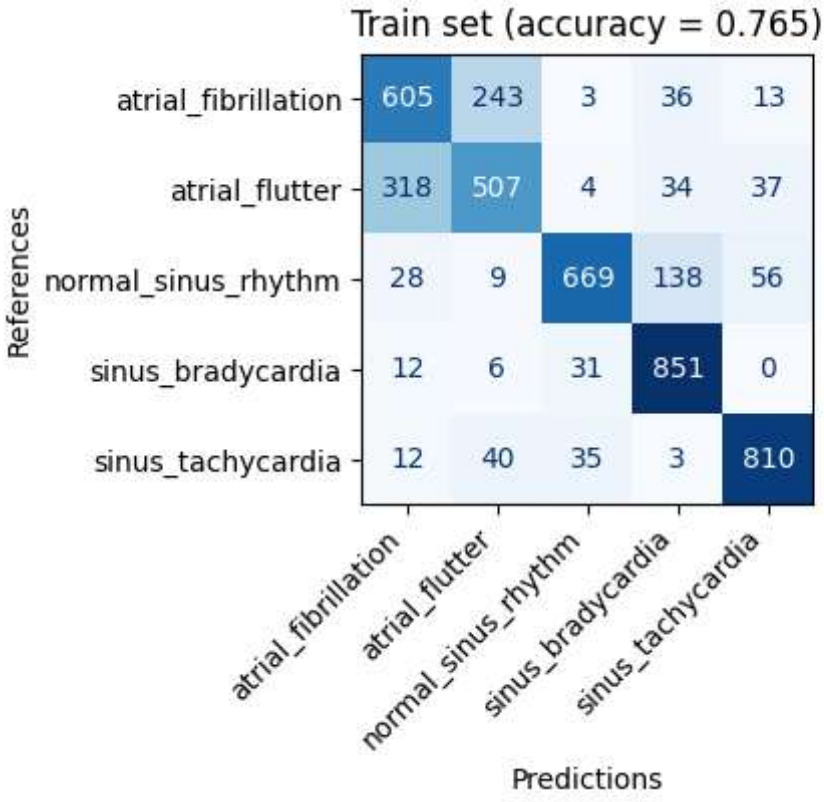
    if labels is not None:
        plt.xticks(np.arange(c.shape[-1]), labels, rotation=45, ha='right')
        plt.yticks(np.arange(c.shape[-1]), labels)
    plt.xlabel('Predictions')
    plt.ylabel('References')
    if title is not None:
        plt.title(title)



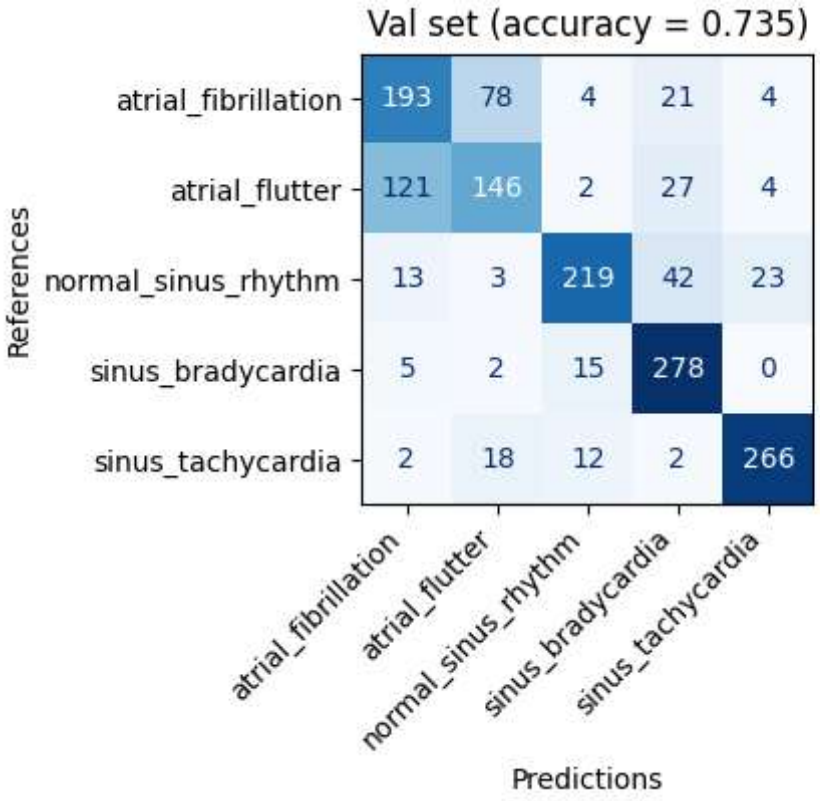
def plot_confusion_matrices(matrices, labels):
    for subset in ('train', 'val', 'test'):
        c = matrices[subset]
        accuracy = np.trace(c) / c.sum()
        title = f'{subset.capitalize()} set (accuracy = {accuracy:.3f})'
        plot_confusion_matrix(c, labels=labels, title=title)


plot_confusion_matrices(cnn_matrices, np.unique(rhythms))
```

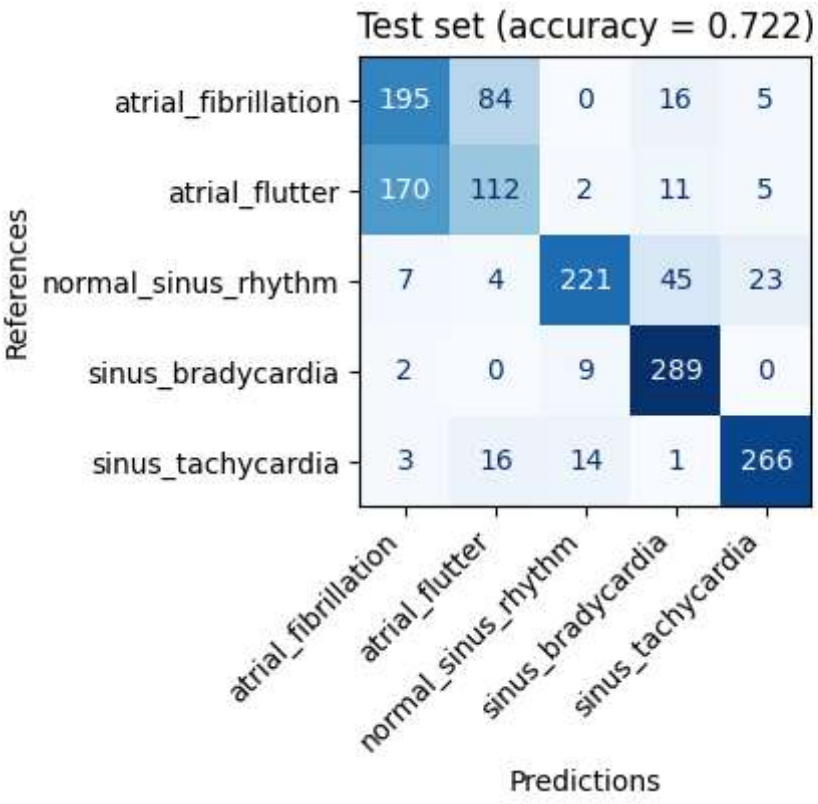
Figure



Figure



Figure



Question 2

Comment the metrics shown in TensorBoard and the confusion matrices. Does the model overfit? Are there some rhythms that are difficult to classify?

Answer: The CNN reaches ~0.76 train / 0.73 val / 0.72 test accuracy, so overfitting is mild. Confusion matrices show most errors between atrial fibrillation and atrial flutter, and some sinus tachycardia beats mapped to normal sinus (and vice versa); bradycardia is classified reliably. Overall, performance is good but atrial arrhythmias remain harder to separate.

Question 3

Define two custom models to classify cardiac rhythms from ECG signals.

You can directly define the layers of the custom models in the following classes.

In [11]:
```
class CustomModel1(torch.nn.Module):

    def __init__(self, input_shape, output_shape):
        super().__init__()
        self.input_shape = input_shape
        self.output_shape = output_shape

        # Implement you own model here.
        self.layers = torch.nn.Sequential(
            torch.nn.Flatten(),
            torch.nn.Linear(np.prod(self.input_shape), self.output_shape[0]),
        )

    def forward(self, x):
```

```
        return self.layers(x)


class CustomModel2(torch.nn.Module):

    def __init__(self, input_shape, output_shape):
        super().__init__()
        self.input_shape = input_shape
        self.output_shape = output_shape

        # Implement you own model here.
        self.layers = torch.nn.Sequential(
            torch.nn.Flatten(),
            torch.nn.Linear(np.prod(self.input_shape), self.output_shape[0]),
        )

    def forward(self, x):
        return self.layers(x)
```

You can train and evaluate the first custom model.

In [12]:
```
custom1 = train_model(
    name='custom1',
    model=CustomModel1(input_shape, output_shape),
    x=signals,
    y=encoded_rhythms,
    i_train=i_train,
    i_val=i_val,
    learning_rate=0.0001,
    batch_size=batch_size,
    n_epochs=n_epochs,
)

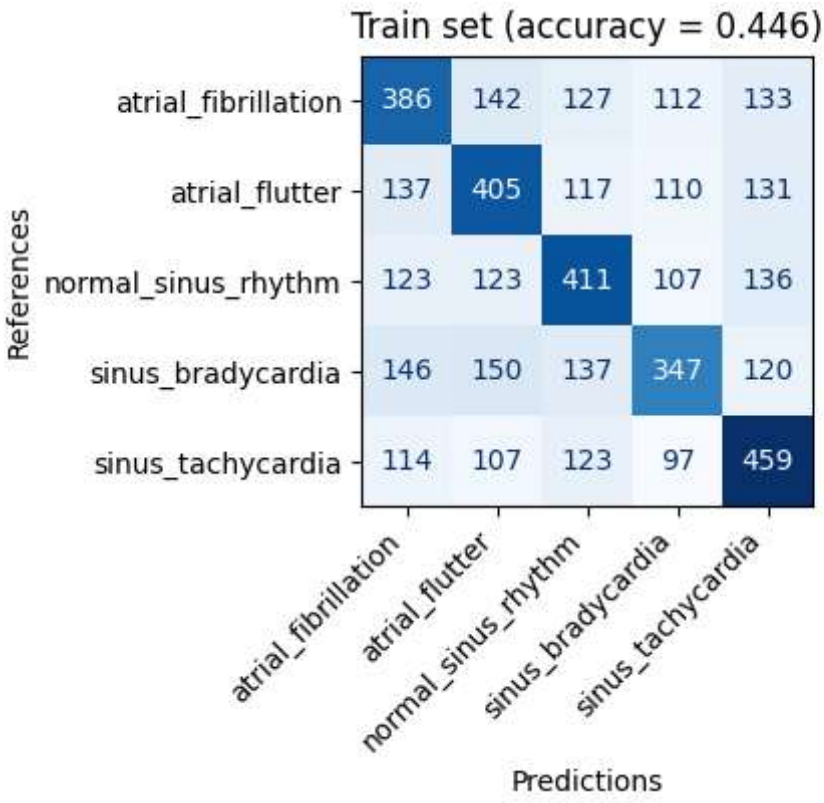
custom1_matrices = evaluate_model(
    model=custom1,
    x=signals,
    y=encoded_rhythms,
    i_train=i_train,
    i_val=i_val,
    i_test=i_test,
    batch_size=batch_size,
)

plot_confusion_matrices(custom1_matrices, np.unique(rhythms))
```

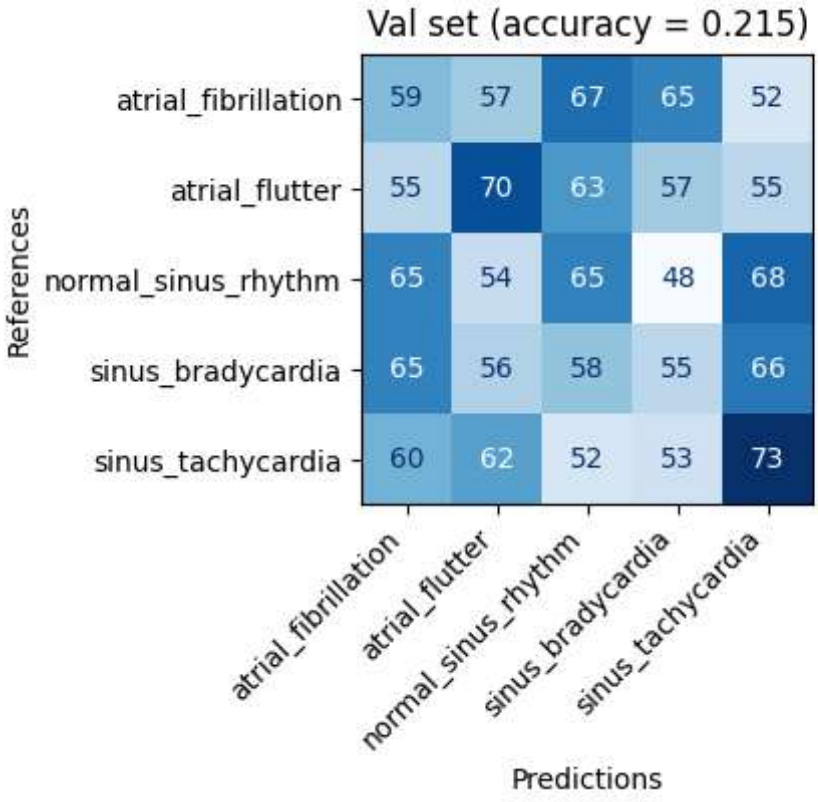
Output()

```
  | Name           | Type         | Params | Mode  | FLOPs  | In sizes     | Out
sizes
-------------------------------------------------------------------------------
-----------
0 | model          | CustomModel1 | 6.4 K  | train | 12.8 K | [1, 1, 1280] | [1,
5]
1 | model.layers   | Sequential   | 6.4 K  | train | 12.8 K | [1, 1, 1280] | [1,
5]
2 | model.layers.0 | Flatten      | 0      | train | 0      | [1, 1, 1280] | [1,
1280]
3 | model.layers.1 | Linear       | 6.4 K  | train | 12.8 K | [1, 1280]    | [1,
5]
-------------------------------------------------------------------------------
-----------
6.4 K     Trainable params
0         Non-trainable params
6.4 K     Total params
0.026     Total estimated model params size (MB)
4         Modules in train mode
0         Modules in eval mode
12.8 K    Total Flops
```

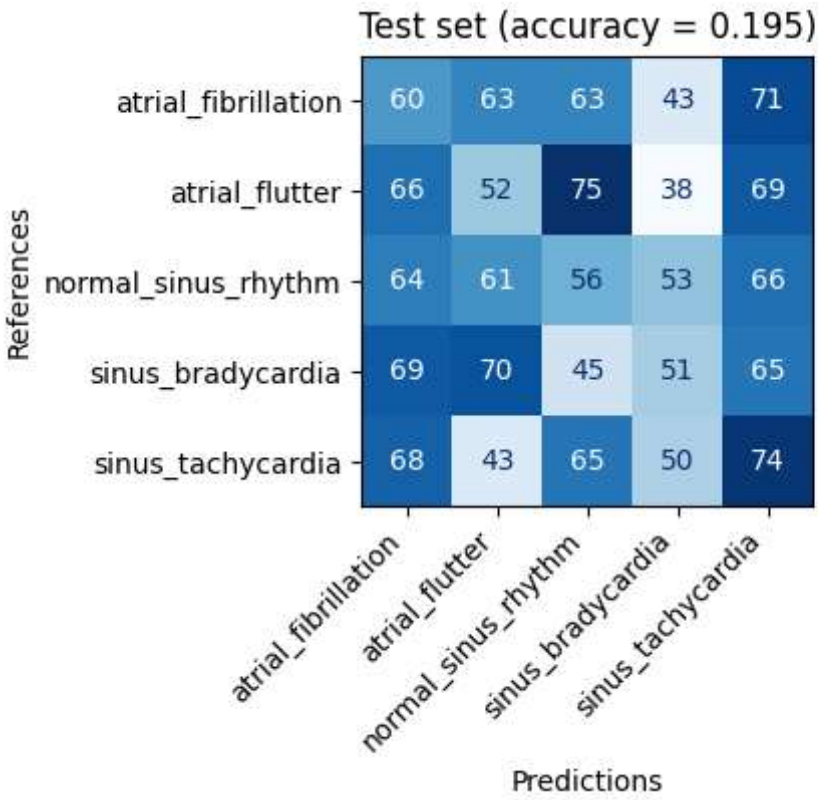
Figure



Figure



Figure



And then the second custom model.

In [13]:
```
custom2 = train_model(
    name='custom2',
    model=CustomModel2(input_shape, output_shape),
    x=signals,
    y=encoded_rhythms,
```

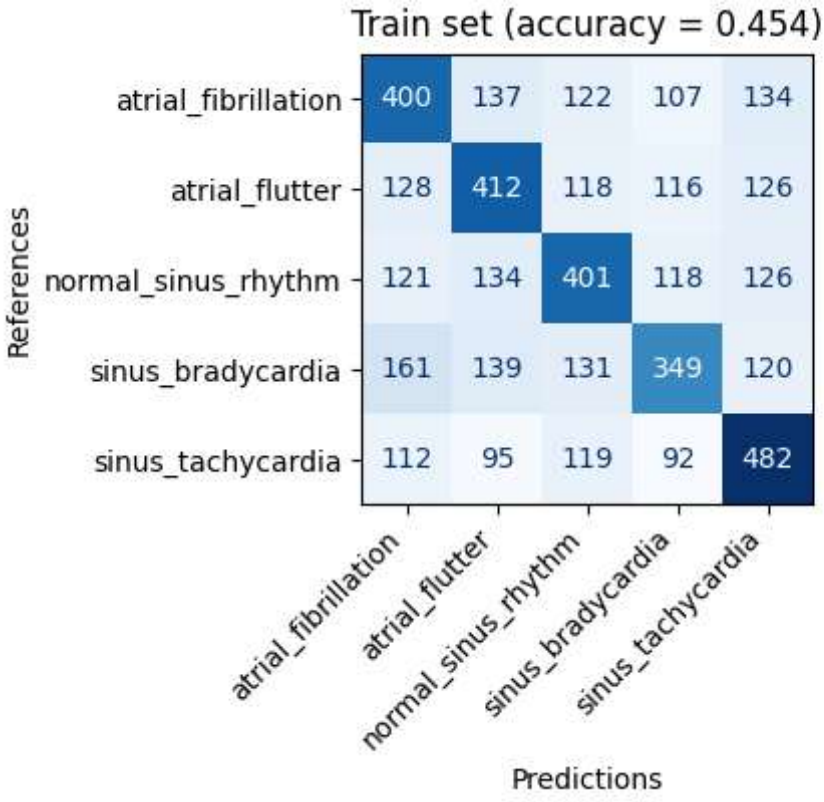
```
    i_train=i_train,
    i_val=i_val,
    learning_rate=0.0001,
    batch_size=batch_size,
    n_epochs=n_epochs,
)

custom2_matrices = evaluate_model(
    model=custom2,
    x=signals,
    y=encoded_rhythms,
    i_train=i_train,
    i_val=i_val,
    i_test=i_test,
    batch_size=batch_size,
)

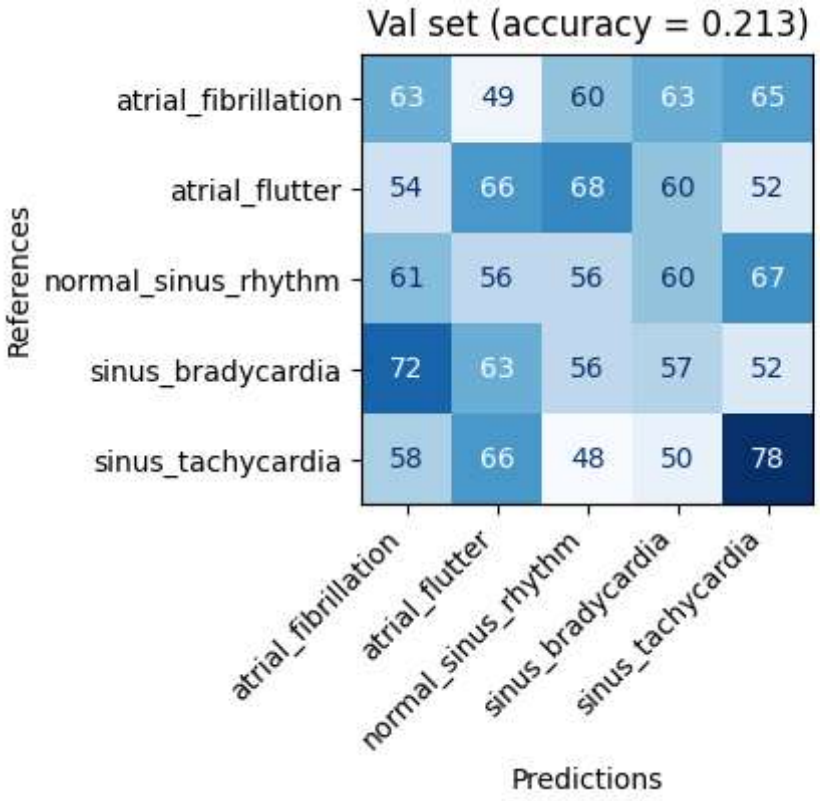
plot_confusion_matrices(custom2_matrices, np.unique(rhythms))
```

```
Output()
  | Name           | Type         | Params | Mode  | FLOPs  | In sizes     | Out
sizes
-------------------------------------------------------------------------------
-----------
0 | model          | CustomModel2 | 6.4 K  | train | 12.8 K | [1, 1, 1280] | [1,
5]
1 | model.layers   | Sequential   | 6.4 K  | train | 12.8 K | [1, 1, 1280] | [1,
5]
2 | model.layers.0 | Flatten      | 0      | train | 0      | [1, 1, 1280] | [1,
1280]
3 | model.layers.1 | Linear       | 6.4 K  | train | 12.8 K | [1, 1280]    | [1,
5]
-------------------------------------------------------------------------------
-----------
6.4 K     Trainable params
0         Non-trainable params
6.4 K     Total params
0.026     Total estimated model params size (MB)
4         Modules in train mode
0         Modules in eval mode
12.8 K    Total Flops
```

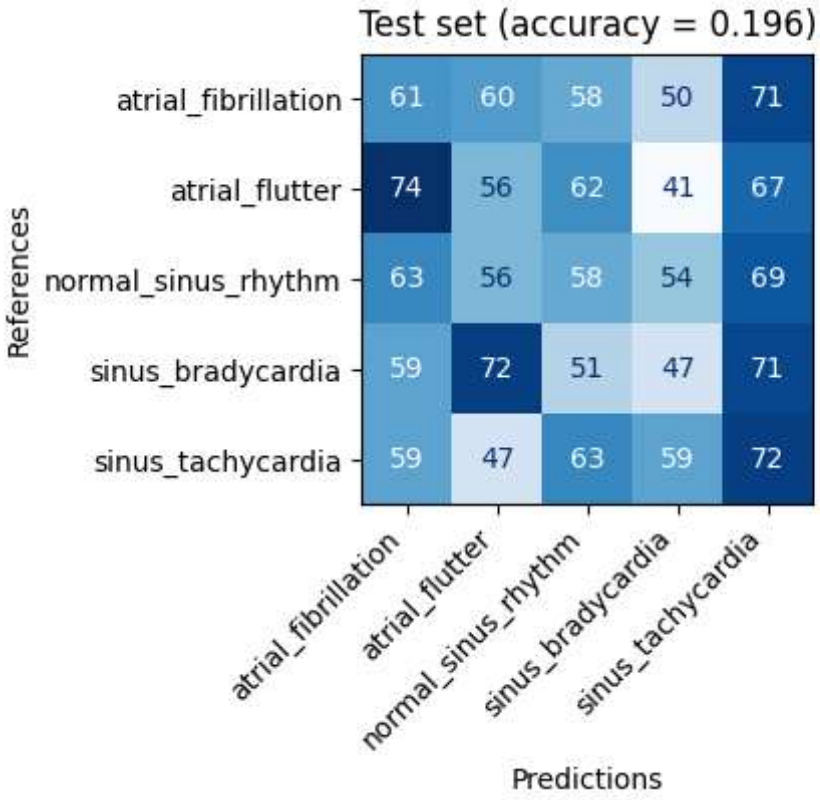
Figure



Figure



Figure



Question 4

How do the two custom models perform? Do they overfit? Do they outpeform the first model?

Answer: Both custom models perform poorly: train acc ~ 0.45 with val/test acc ~ 0.21, so they largely underfit and do not outperform the baseline CNN (val/test ~ 0.73/0.72). They fail to generalize and are far from the first model's performance.